

Data Engineering & Data Science Reference

A study reference covering data flow, tools, and key definitions across data engineering, data science, and analytics roles.

Data Flow by Role

Data Engineer

Builds the pipelines that move data from sources into storage and structured systems.

Multiple Devices/Sources
→ Data Lake (Raw, Unstructured)
→ Data Warehouse (Structured)

Machine Learning Engineer / Data Scientist (EDA)

Pulls from either raw or structured storage to explore and model data.

Data Lake (Raw, Unstructured) —+
|→ EDA / Machine Learning
Data Warehouse (Structured) —+

Data Analyst / BI Analyst

Works primarily with cleaned, structured data to produce insights and visualizations.

Data Warehouse (Structured)
→ Visualizations + EDA/ML
→ Business Insights

Tools & Resources

Datasets and Competitions

- **Kaggle** — datasets, competitions, notebooks, and community projects.

Data Lakes (Raw, Unstructured Storage)

- **Apache Kafka** — distributed event streaming platform
- **Azure Data Lake** — Microsoft's cloud data lake service
- **Amazon S3** — object storage commonly used as a data lake foundation
- **Hadoop (HDFS)** — open source distributed file system

Data Warehouses (Structured, Queryable Storage)

- **Amazon Athena** — serverless query service for S3 data
- **Amazon Redshift** — fully managed cloud data warehouse
- **Google BigQuery** — serverless, highly scalable data warehouse
- **Snowflake** — cloud-native data warehouse with separated compute/storage
- **Azure Synapse Analytics** — Microsoft's unified analytics platform

Data Processing Frameworks

- **Apache Spark** — in-memory distributed processing
 - **Apache Flink** — stream-first processing engine
 - **Hadoop MapReduce** — batch processing (legacy but still widely used)
-

Learning Resources

SQL

- **Khan Academy SQL** — intro to SQL
- **Mode SQL Tutorial** — mode.com/sql-tutorial
- **SQLZoo** — interactive SQL practice

General Data Engineering

- **DataCamp** — structured courses in SQL, Python, and data engineering
 - **Coursera / edX** — university-backed data engineering specializations
-

Definitions

ETL (Extract, Transform, Load)

The foundational data pipeline pattern used to move data into a warehouse.

- **Extract** — pull data from a source (such as a data lake, API, or operational database).
- **Transform** — clean, enrich, validate, and organize the data into a useful state.
- **Load** — pipe the transformed data into a data warehouse where it can be queried and reused.

A modern variation is **ELT** (Extract, Load, Transform), where raw data is loaded into the warehouse first and transformed afterward using the warehouse's compute power. ELT is common with cloud warehouses like BigQuery and Snowflake.

Tags: [Data Pipeline](#) · [Data Engineer](#) · [ETL](#) · [ELT](#)

Relational Database

A database that stores data in tables with rows and columns, where relationships between tables are enforced through keys.

- Uses **SQL** (Structured Query Language) to query and manipulate data.
- Provides **ACID guarantees** (Atomicity, Consistency, Isolation, Durability) for reliable transactions.
- Requires a **predefined schema** — tables, columns, and data types must be designed before data is inserted.
- Strong on consistency and data integrity, but harder to scale horizontally across distributed systems.
- Common examples: **PostgreSQL**, **MySQL**, **SQL Server**, **Oracle**, **SQLite**.

Tags: [Database](#) · [Relational Database](#) · [SQL](#) · [PostgreSQL](#) · [MySQL](#) · [ACID](#)

Non-Relational Database (NoSQL)

A database that stores data in formats other than traditional tables, typically without requiring a predefined schema.

- Allows flexible, schemaless data storage — similar to organizing files in folders without a strict structure upfront.
- Designed to scale horizontally across many machines.
- Typically trades some consistency guarantees for availability and partition tolerance (see **CAP theorem**).
- **Document stores:** MongoDB, CouchDB
- **Key-value stores:** Redis, Riak
- **Wide-column stores:** Cassandra, Apache HBase, Hypertable
- **Graph databases:** Neo4j, Amazon Neptune

Tags: [NoSQL](#) · [MongoDB](#) · [Redis](#) · [Cassandra](#) · [CouchDB](#) · [HBase](#) · [Document Store](#) · [Key-Value Store](#)

Hadoop

An open source distributed processing framework originally developed by Yahoo! for handling big data workloads.

- **HDFS (Hadoop Distributed File System)** — stores and scales data across many machines, providing fault tolerance through replication.
- **MapReduce** — a batch processing model that breaks jobs into map and reduce phases, running them in parallel across a cluster.
- **YARN (Yet Another Resource Negotiator)** — the cluster resource manager that schedules jobs.
- **Hive** — write SQL-like queries against HDFS data, even though HDFS isn't a relational database.
- **Pig** — scripting language for data transformations on Hadoop.
- **HBase** — NoSQL database built on top of HDFS.

Hadoop is often considered the foundation of big data infrastructure, though modern workloads frequently use Spark instead of MapReduce for faster processing.

Tags: [Hadoop](#) · [HDFS](#) · [MapReduce](#) · [Hive](#) · [YARN](#) · [Big Data](#) · [Data Lake](#)

Apache Spark

An open source distributed processing engine designed for large-scale data analytics.

- **In-memory processing** — significantly faster than Hadoop MapReduce because intermediate results stay in RAM instead of being written to disk.
- Handles both **batch processing** and **real-time streaming**, unlike Hadoop which is primarily batch-oriented.
- High-level APIs in **Python (PySpark)**, **Scala**, **Java**, and **R**.
- **Spark SQL** — structured data queries using SQL or DataFrame APIs.
- **Spark Streaming** — real-time and near-real-time data processing.
- **MLlib** — machine learning at scale with algorithms for classification, regression, clustering, and more.
- **GraphX** — graph computation and network analysis.
- Runs on Hadoop/HDFS, cloud storage (S3, Azure Data Lake, GCS), or standalone clusters.
- Commonly orchestrated through **Databricks**, **Amazon EMR**, or **Google Dataproc**.

Tags: [Apache Spark](#) · [PySpark](#) · [Big Data](#) · [Distributed Computing](#) · [Spark SQL](#) · [MLlib](#) · [Stream Processing](#) · [Databricks](#)

Apache Flink

An open source distributed processing engine built for stateful computations over data streams, with a stream-first design philosophy.

- **True streaming** — processes each event as it arrives with very low latency, rather than treating streams as fast micro-batches (the Spark Streaming approach).
- **Stateful processing** — maintains and manages application state across events, with built-in checkpointing for fault tolerance.
- **Event-time processing** — handles out-of-order events correctly using watermarks and event timestamps.
- Supports both **bounded (batch)** and **unbounded (streaming)** datasets using the same APIs.
- **DataStream API** — the primary API for unbounded stream processing.
- **Table API / Flink SQL** — higher-level relational API for both streaming and batch.
- **CEP (Complex Event Processing)** — library for detecting patterns across event streams.

When Flink vs Spark: Choose Flink for true low-latency streaming, event-time semantics, or heavy stateful processing. Choose Spark for mostly-batch workloads or when you need a unified engine covering SQL, ML, and graph processing.

Tags: [Apache Flink](#) · [Stream Processing](#) · [Real-Time Data](#) · [Stateful Processing](#) · [Event-Time](#) · [Distributed Computing](#)

Data Lake

A centralized storage repository that holds vast amounts of raw data in its native format until it's needed.

- Stores **structured, semi-structured, and unstructured** data (CSV, JSON, images, logs, video, etc.).
- Schema is applied **on read**, not on write (schema-on-read).
- Cost-effective for storing large volumes of data whose future use may not be fully known.
- Common implementations: Amazon S3, Azure Data Lake Storage, Hadoop HDFS.

Tags: [Data Lake](#) · [S3](#) · [HDFS](#) · [Schema-on-Read](#) · [Unstructured Data](#)

Data Warehouse

A centralized repository optimized for querying and analyzing structured data.

- Data is **cleaned, transformed, and modeled** before loading (schema-on-write).
- Optimized for fast analytical queries (**OLAP**) across large historical datasets.
- Typically used as the source for BI dashboards, reports, and analytics.
- Common implementations: Amazon Redshift, Google BigQuery, Snowflake, Azure Synapse.

Tags: [Data Warehouse](#) · [OLAP](#) · [Redshift](#) · [BigQuery](#) · [Snowflake](#) · [Schema-on-Write](#)

Apache Kafka

A distributed event streaming platform used for building real-time data pipelines and streaming applications.

- Publishes and subscribes to streams of records, similar to a message queue but far more scalable.
- Stores streams durably and reliably across a cluster.
- Processes streams as they occur.
- Often sits at the front of a data pipeline, ingesting events from applications before they land in a data lake or warehouse.

Tags: [Apache Kafka](#) · [Streaming](#) · [Event Pipeline](#) · [Real-Time Data](#)

Quick Reference: Which Tool for Which Job?

Need	Consider
Store raw, diverse data cheaply	Data lake (S3, Azure Data Lake, HDFS)
Run fast analytical queries on structured data	Data warehouse (BigQuery, Redshift, Snowflake)
Process huge datasets in parallel	Apache Spark
Stream events in real time	Apache Kafka, Spark Streaming, Flink
Transactional application database	Relational DB (PostgreSQL, MySQL)
Flexible schema, high write volume	NoSQL (MongoDB, Cassandra)
Query unstructured data in S3	Amazon Athena
SQL-like queries on Hadoop	Hive